WEEK 10 - PYTHON PROGRAMMING

From Proposal to Design

Your proposal says what. Today we decide how.

ACT 1 · DECOMPOSE
split

ACT 2 · CONTRACTS
promise

ACT 3 · MODULES
organize

LAB
design yours

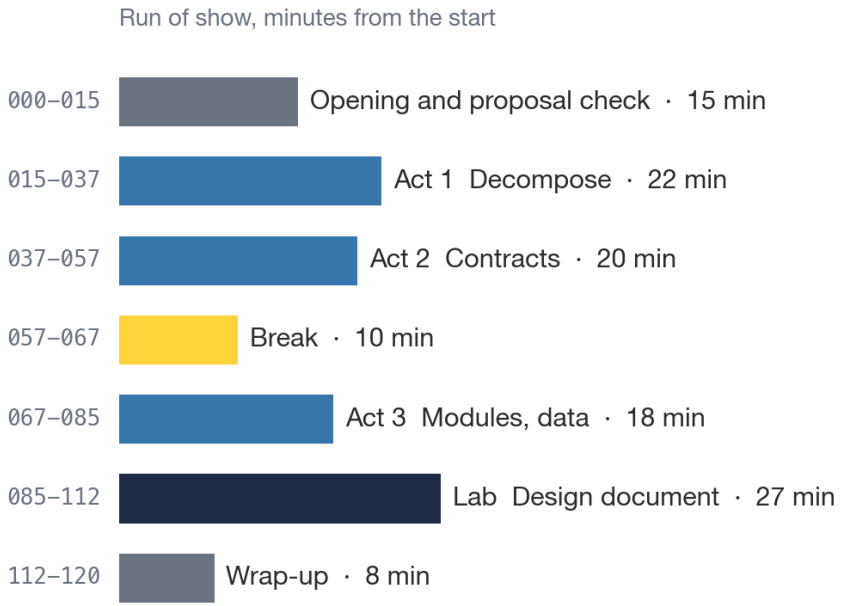
120 minutes, including a break

Instructor: MinGi KYUNG

The Plan: Draw It Before You Build It

Last week you chose a problem. This week you sketch the program before you type it.

- Decompose: stories become features, features become functions.
- Contracts: every function promises its inputs, output and failures.
- Modules and data: one file per job, one shape per kind of data.
- Lab: your own design document, reviewed by a classmate.

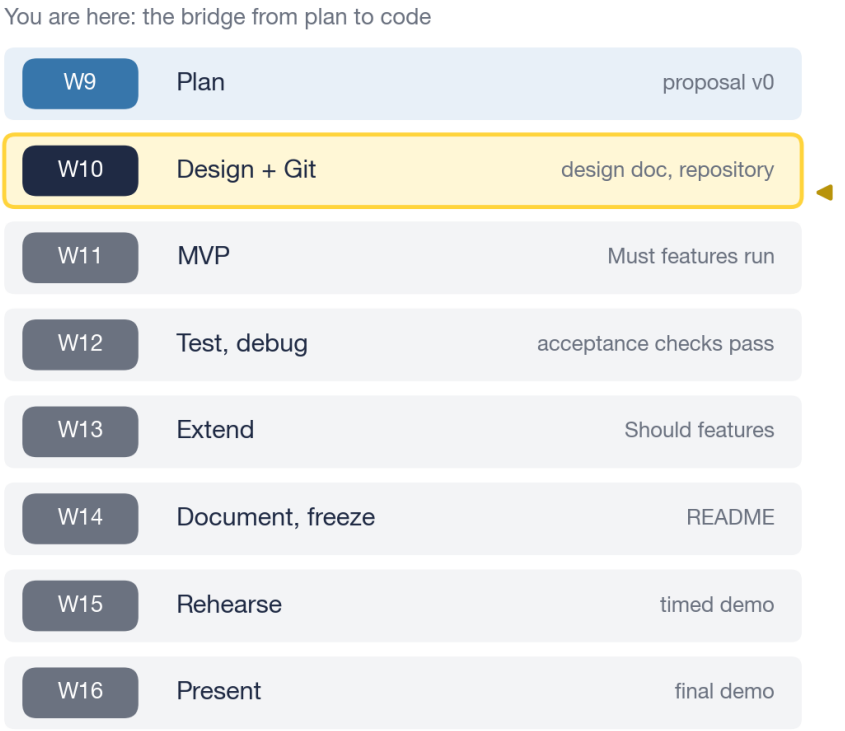


Where We Are in the Project

Week 9 gave you a proposal. Week 11 needs a running MVP. Week 10 is the bridge.

- Week 9: problem, user stories, Must features, acceptance criteria.
- Week 10: design document today, Git repository next session.
- Week 11: the Must features run end to end.

RULE Must scope freezes at the end of this week. New ideas go to Should or Could.



Check Your Proposal First

Five minutes. Design is built on the proposal, so cracks spread.

- At most 3 Must features, each one sentence.
- Each Must feature has a pass/fail acceptance criterion.
- Your data file has a name, columns and types.
- Your Won't list has at least two items.
- Missing something? Write it now, in pencil.

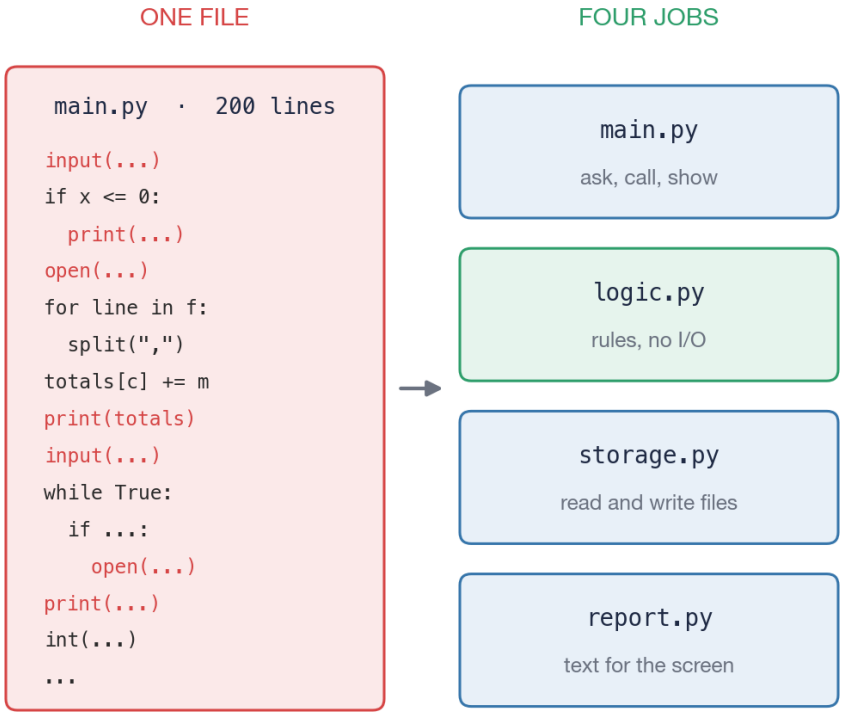
> **TIP** A criterion like "entering -30 minutes prints a warning; the file is unchanged" is gold. It becomes a test in week 12.

The 200-Line main.py

Every project starts as one file. Most stay that way, and that is the problem.

- You cannot test the totals without typing at the keyboard.
- A change to the printing breaks the arithmetic.
- In week 14 you cannot find anything, and neither can the grader.
- The cure is not more code. It is a plan for where code lives.

PREDICT Which of these two would you rather debug the night before your demo?



Design Means Decisions on Paper

A box on paper costs a minute to move. The same change in code costs an evening.

- Which functions exist, and what is each one's single job?
- What does each function take, return, and do when input is bad?
- Which file holds each function?
- What does one record of data look like, in memory and on disk?

PAYOFF Your design document becomes the week 11 checklist. Implement one contract, tick one box.

ACT 1 · DECOMPOSE

Decompose: From Stories to Functions

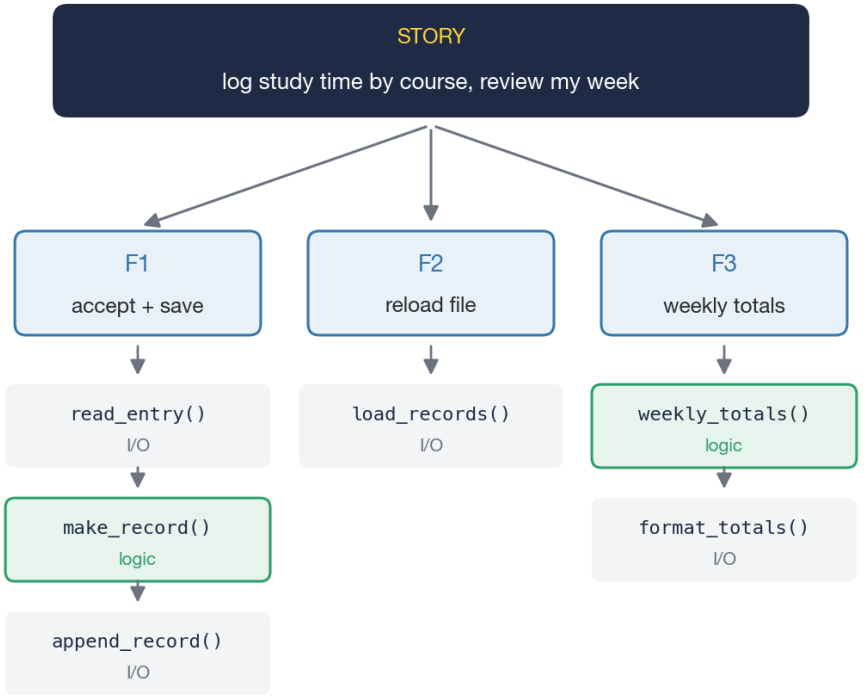
Big problems are small problems standing in a line.

- A story says why. A feature says what. A function says how.
- We follow one feature of the study logger from sentence to code.

Story, Feature, Function

Three levels. Each level splits into a few pieces of the next.

- Story: As a student, I want to log study time by course, so that I can review my week.
- Feature: F1 accepts date, course and positive minutes and saves valid entries.
- Function: one verb, one job, testable alone.
- Aim for 2-4 functions per Must feature.



story = why · feature = what · function = how

The Verb Test

Read the feature sentence aloud. The verbs are your functions.

- F1: accept date, course and minutes; check the minutes; save valid entries.
- Verbs become functions: read, check, save.
- Nouns become data: date, course, minutes, record.
- Name functions verb_noun: make_record, not record_stuff.

TIP If a function name needs "and", it is two functions.

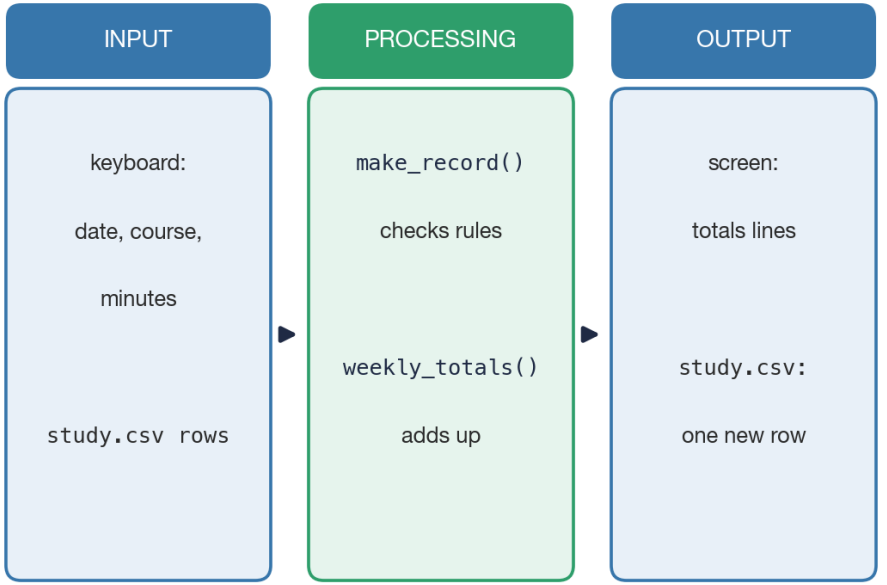
OUTPUT

```
F1  accept, check and save one entry
    -> read_entry()      asks the user      I/O
    -> make_record()     checks and builds  logic
    -> append_record()   writes the file    I/O
```

Input, Processing, Output

Every feature is a pipeline. Draw it before you code it.

- Input: what arrives from the keyboard or a file.
- Processing: rules and calculations on that input.
- Output: what goes to the screen or back to a file.
- Processing is where the bugs live, and where the marks are.



bugs and marks live in the middle column

TRAP: One Function Does Everything

This works, until the second day of use.

- To test the totals you must type three answers every time.
- The CSV header row crashes int() on the next run.
- Reading the code, would you have seen that bug?

OUTPUT

```
ValueError: invalid literal for int()
with base 10: 'minutes'
the header row reached int()
```

PYTHON

```
def run():
    date = input("Date: ")
    course = input("Course: ").strip().lower()
    minutes = int(input("Minutes: "))
    if minutes <= 0:
        print("Not saved")
        return
    with open("study.csv", "a") as f:
        f.write(f"{date},{course},{minutes}\n")
    totals = {}
    for line in open("study.csv"):
        d, c, m = line.strip().split(",")
        totals[c] = totals.get(c, 0) + int(m)
    print(totals)

run()
```

Split by Job

Same rules, now in functions with no input(), print() or open().

- parse_minutes checks one value.
- weekly_totals adds up a list of records.
- Neither knows where data comes from or goes to.
- So you can call them with any data you like.

PYTHON

```
def parse_minutes(text):
    minutes = int(text)
    if minutes <= 0:
        raise ValueError("minutes must be positive")
    return minutes

def weekly_totals(records):
    totals = {}
    for r in records:
        c = r["course"]
        totals[c] = totals.get(c, 0) + r["minutes"]
    return totals

rows = [{"course": "python", "minutes": 90},
        {"course": "stats", "minutes": 45}]
print(weekly_totals(rows))
```

OUTPUT

```
{'python': 90, 'stats': 45}
```

Test Without Typing

Logic functions can be checked in one second, as often as you like.

- Each assert is one acceptance criterion from your proposal.
- No keyboard, no files, no waiting.
- Run the checks after every change.

PAYOFF Week 9's "entering -30 prints a warning" just became a check.

PYTHON

```
def parse_minutes(text):
    minutes = int(text)
    if minutes <= 0:
        raise ValueError("minutes must be positive")
    return minutes

assert parse_minutes("90") == 90
for bad in ["-30", "0", "abc"]:
    try:
        parse_minutes(bad)
        print("accepted", bad)
    except ValueError:
        pass
print("All checks passed.")
```

OUTPUT

```
All checks passed.
```

Practice 1: Decompose Your F1

ACT 1 · DECOMPOSE

Five minutes, on paper or in a text file. Your project, not the study logger.

- Write your F1 as one sentence.
- Underline the verbs. One function per verb.
- Name each function verb_noun.
- Mark each one I/O or logic.

TIP Stuck? Ask three questions: what does the user give, what must be checked, what must be kept?

OUTPUT

```
F1  <your feature sentence>
    -> verb_noun()      what it does      I/O
    -> verb_noun()      what it does      logic
    -> verb_noun()      what it does      I/O
```

ACT 2 · CONTRACTS

Contracts: Promises Between Functions

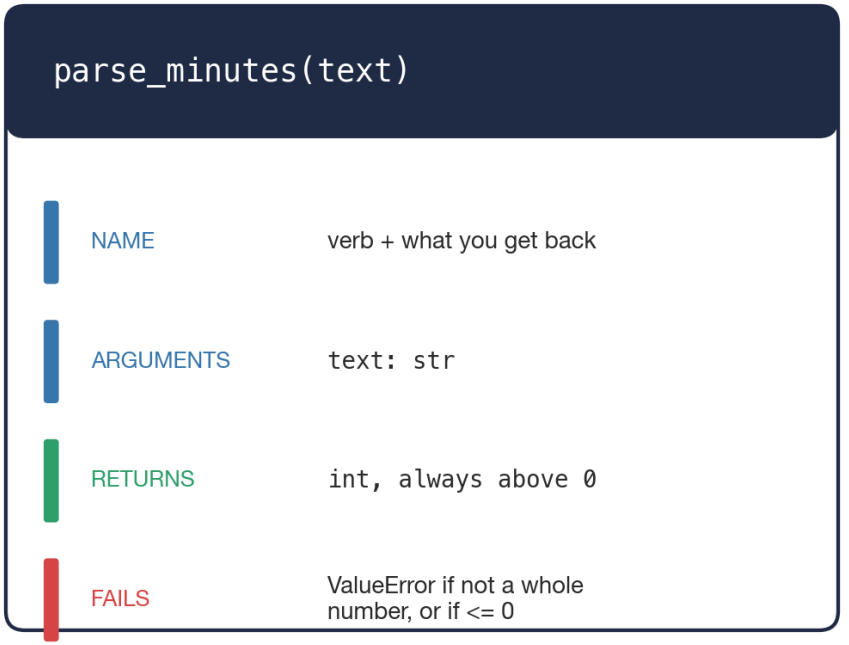
Give me this, and I give you that, or I tell you why not.

- A contract is decided before the body is written.
- It is the part of your design that a grader can check line by line.

Anatomy of a Contract

Four fields. If you cannot fill all four, the function is not designed yet.

- Name: a verb that says what you get back.
- Arguments: names and types.
- Return value: one type, always the same type.
- Failure: what happens with bad input.



Write the Contract in the Docstring

The docstring is the contract, stored beside the code it describes.

- First line: what the function returns.
- Then: what makes it fail, and how.
- VS Code shows the docstring when you hover over a call.
- Write the docstring first. The body comes in week 11.

PYTHON

```
def parse_minutes(text):
    """Return text as a positive int of minutes.

    Raises ValueError if text is not a whole
    number or is not above 0.
    """
    minutes = int(text)
    if minutes <= 0:
        raise ValueError("minutes must be positive")
    return minutes

print(parse_minutes("45"))
print(parse_minutes.__doc__.splitlines()[0])
```

OUTPUT

```
45
Return text as a positive int of minutes.
```

Choose One Failure Style

Bad input will arrive. Decide on paper what each function does with it.

- raise ValueError: the input breaks a rule. The caller must react.
- return None: "nothing found" is a normal answer.
- print inside: avoid. The caller cannot react and tests cannot see it.

TRAP A function that sometimes returns an int and sometimes returns "error" breaks every caller.

PYTHON

```
def find_minutes(totals, course):
    """Return the minutes, or None if absent."""
    return totals.get(course)

totals = {"python": 90}
print(find_minutes(totals, "python"))
print(find_minutes(totals, "art"))
```

OUTPUT

```
90
None
```

Logic Raises, main Talks

The rule lives in logic. The message for the user lives at the edge.

- parse_minutes only raises. It never prints.
- The caller catches ValueError and decides what the user sees.
- The same parse_minutes works in a menu, a test or a chart script.

PYTHON

```
def parse_minutes(text):
    minutes = int(text)
    if minutes <= 0:
        raise ValueError("minutes must be positive")
    return minutes

for typed in ["90", "-30", "abc"]:
    try:
        minutes = parse_minutes(typed)
    except ValueError as error:
        print("Not saved:", error)
    else:
        print("Saved", minutes)
```

OUTPUT

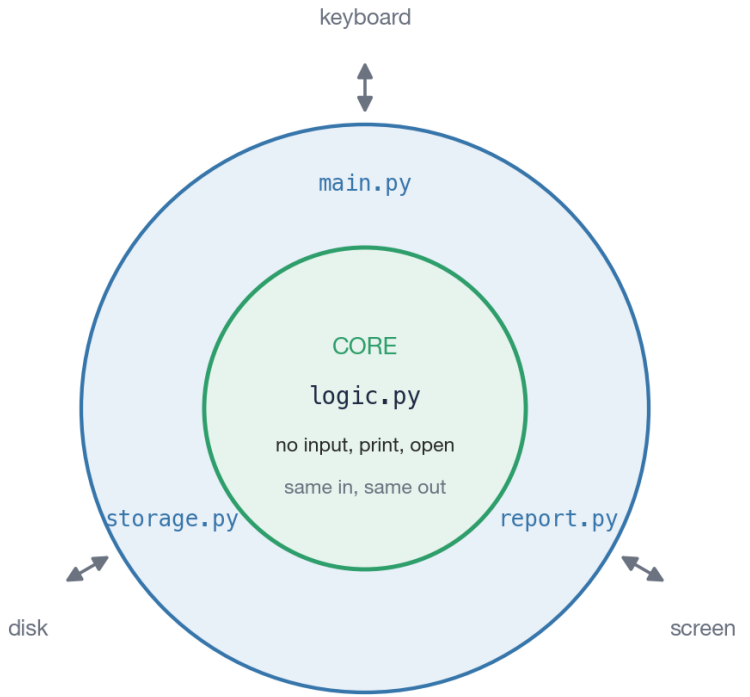
```
Saved 90
Not saved: minutes must be positive
Not saved: invalid literal for int() with base 10: 'abc'
```

Keep I/O at the Edges

A pure core in the middle, a thin shell of input and output around it.

- Core: logic.py. Same input, same output, every time.
- Shell: main.py, storage.py, report.py talk to keyboard, disk and screen.
- The core never calls input(), print() or open().
- Bugs in the core show up in tests. Bugs in the shell show up in the demo.

PAYOFF In week 12 you test the core with asserts and the shell by running it once.



SHELL: talks to the world

main() Reads Like a Table of Contents

main() calls. It does not calculate.

- Each line is a step from the design: ask, build, save, total, show.
- Reading main() tells the whole story in ten lines.
- If main() grows past about 20 lines, a function is missing.

PYTHON

```
def main():
    date = input("Date (YYYY-MM-DD): ")
    course = input("Course: ")
    minutes = input("Minutes: ")
    try:
        record = make_record(date, course, minutes)
    except ValueError as error:
        print("Not saved:", error)
    else:
        append_record(DATA, record)
        print("Saved.")
    totals = weekly_totals(load_records(DATA))
    for line in format_totals(totals):
        print(line)
```

Practice 2: Split the Monolith

Ten minutes. A new story: an expense tracker squeezed into one function.

- Open labs/session1/refactor_lab.py.
- Read all_in_one(). Do not change it.
- Complete the three TODO functions. Each docstring is the contract.
- Run the file until all six checks print PASS.

TIP " food " and "Food" are the same category. strip() and lower() before you add.

OUTPUT

```
PASS parse_amount('4500')
PASS parse_amount rejects '-200'
PASS parse_amount rejects 'ten'
PASS category_totals(sample)
PASS category_totals([])
PASS format_report
```

BREAK

Halftime

Ten minutes. Stretch, refill, then we decide where each function lives.

- After the break: modules, imports and the shape of your data.
- Then the lab: your own design document.

ACT 3 · MODULES

Modules and Data: Where Things Live

One file per responsibility. One shape per kind of data.

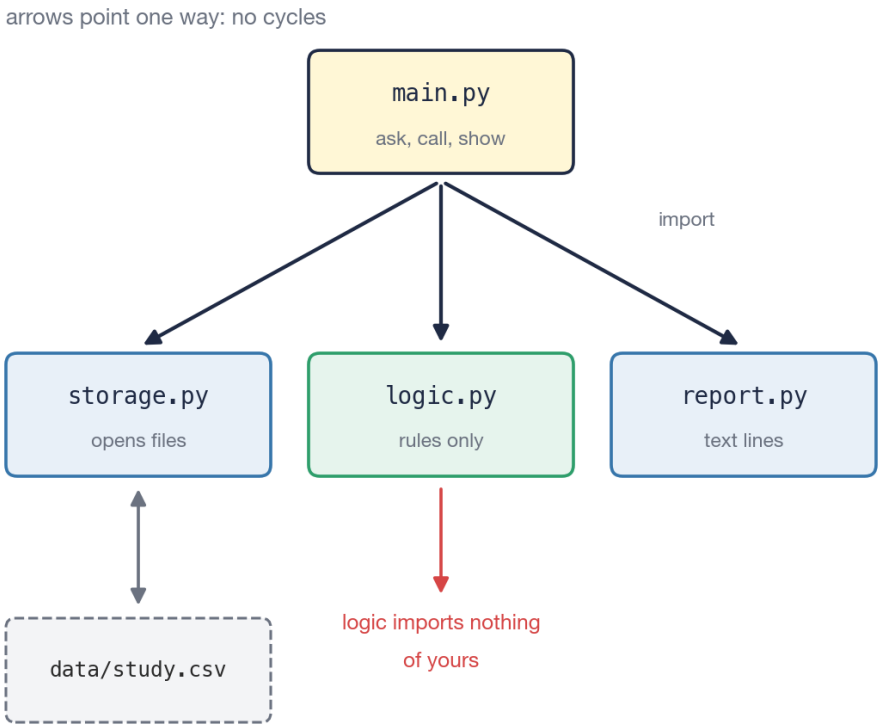
- A module is just a .py file you import.
- Your design decides which function goes in which file.

One Responsibility per File

Four files, four jobs. Every function has exactly one home.

- main.py runs the program: ask, call, show.
- logic.py holds the rules. No I/O at all.
- storage.py is the only file that opens data files.
- report.py turns results into lines of text.

> **RULE** Imports point one way. logic.py imports none of your other files.



Importing Your Own Files

A file in the same folder is a module. Its name is the file name without .py.

- from logic import weekly_totals loads logic.py once.
- Import names, not everything: avoid import *.
- Run python3 from the project folder so Python finds the files.

PYTHON

```
from logic import make_record, weekly_totals
from report import format_totals

rows = [make_record("2026-10-24", "python", "90"),
        make_record("2026-10-25", "Stats", "45")]
for line in format_totals(weekly_totals(rows)):
    print(line)
```

OUTPUT

```
python    90 min
stats     45 min
```

Run It or Import It

if __name__ == "__main__" separates "run this file" from "lend me your functions".

- Run python3 report.py: the block runs.
- import report from another file: the block is skipped.
- Use it for a quick self-check at the bottom of a module.

PYTHON

```
def format_totals(totals):
    return [f"{c:<8}{m:>4} min"
            for c, m in totals.items()]

if __name__ == "__main__":
    print(format_totals({"python": 90}))
```

OUTPUT

```
['python    90 min']
python3 report.py   -> the check runs
import report       -> silent
```

Two Import Traps

Both cause confusing errors far from the real cause.

- Never name a file after a library: csv.py, random.py.
- Your csv.py hides the real one.
- Never let two of your modules import each other.
- One of them would see a half-built module.

TIP Rename the file and delete __pycache__ if the error stays.

OUTPUT

```
$ python3 main.py
AttributeError: module 'csv' has no
attribute 'DictReader'
cause: your own csv.py in the folder
```

Three Shapes of Data

Decide the shapes now. Every function's contract mentions them.

- One record: a dict with fixed keys.
- All records: a list of dicts, in file order, duplicates allowed.
- A summary: a dict from key to result, one entry per key.

PREDICT Why a list for the records but a dict for the totals?

ONE RECORD — dict, fixed keys

```
{"date": "2026-10-24", "course": "python", "minutes": 90}
```

ALL RECORDS — list of dicts, file order

```
[0]  {"date": "10-24", "course": "python", "minutes": 90}

[1]  {"date": "10-25", "course": "stats", "minutes": 45}

[2]  {"date": "10-26", "course": "python", "minutes": 30}
```

weekly_totals()

SUMMARY — dict: key -> result

```
{"python": 120, "stats": 45}
```

From File to Records

csv.DictReader turns each row into a dict, using the header as keys.

- Every value arrives as a str.
- Convert numbers once, right after loading, in storage.py.
- Then every other function can trust the types.

TRAP "90" + "45" is "9045". Convert before you add.

PYTHON

```
import csv

with open("data/study.csv", newline="") as f:
    rows = list(csv.DictReader(f))
print(rows[0]["course"], repr(rows[0]["minutes"]))
for r in rows:
    r["minutes"] = int(r["minutes"])
print(sum(r["minutes"] for r in rows))
```

OUTPUT

```
python '90'
135
'90' is a str until int() converts it
```

The Project Folder

This layout becomes your repository in Session 2.

- Code at the top level, data in data/.
- One test file for the core.
- README and .gitignore from the first commit.
- The full example lives in examples/study_logger/.

OUTPUT

```
study-logger/
  main.py          runs the program
  logic.py         rules, no I/O
  storage.py       reads and writes data/
  report.py        text for the screen
  test_logic.py    acceptance checks
  data/
    study.csv      date,course,minutes
  README.md
  .gitignore
```

LAB

Lab: Your Design Document

27 minutes. Your project, on paper. Code comes next session and next week.

- Template: labs/session1/design-document.md.
- Part A: functions and contracts. Part B: modules and data. Part C: review.

Lab Part A: Functions and Contracts

LAB

12 minutes. Sections 1-3 of the template.

- Copy your Must features and acceptance criteria from the proposal.
- Decompose each Must feature into 2-4 functions.
- Write all four contract fields for every function.
- Write main()'s call sequence as numbered steps.

TIP Aim for 4-8 functions in total. More is fine later; fewer means jobs are still mixed.

OUTPUT

```
make_record(date: str, course: str, minutes_text: str)
  returns  dict {date: str, course: str, minutes: int}
  fails    ValueError if course is empty or minutes <= 0
  module   logic.py      feature  F1
```

Lab Part B: Modules and Data

LAB

10 minutes. Sections 4-5 of the template.

- Assign every function to main, logic, storage or report.
- Draw the import arrows. Check that they point one way.
- Write one record as a dict, with real values.
- Write a 3-5 row sample file and its expected output, by hand.

OUTPUT

```
data/study.csv
date,course,minutes
2026-10-24,python,90
2026-10-25,stats,45

expected totals: python 90, stats 45
```

Lab Part C: Design Review

LAB

5 minutes with one classmate. Swap documents, not code.

- Can every Must feature be traced to named functions?
- Does logic.py avoid input(), print() and open()?
- Does every contract state its failure behavior?
- Could you write an assert for each logic function right now?
- Give one strength and one concern. Then revise your own document.

RULE Review only. You do not share functions, files or code.

Five Ways to Ruin a Design

Each of these has cost a student an evening in week 11.

- Function names that are nouns: data(), stuff(), process().
- Contracts with no failure behavior.
- print() and input() inside logic functions.
- One utils.py that holds everything.
- Designing Won't features "just in case".

Exit Check

Answer in one line each, then compare with a neighbour.

- Where do function names come from in the verb test?
- What are the four fields of a contract?
- Why does parse_minutes raise instead of printing?
- Which module is allowed to open data files?
- Why are the records a list but the totals a dict?

Submit and Next Session

Your design is the map. Next session we give it a history.

- Submit labs/session1/design-document.md via the LMS today.
- Bring a laptop where git --version runs.
- Sign in to github.com before class, and remember your password.
- Session 2: your module map becomes a GitHub repository with a first pull request.

> **RULE** Must scope freezes at the end of Session 2.